Cabal DeltaNeutral Vault

Security Review

January 17, 2026 - January 21, 2026

Contents

1. About SBSecurity 3
2. Disclaimer 3
3. Risk classification 3
4. Executive Summary 4
5. Findings 5

Medium Severity 7

[M-01] echelon_helper.move#left_to_target_ltv() function returns inverted rebalance action 7
[M-02] delta.move#repay_debt_from_lp_pool_and_withdraw() sanity check makes partial withdrawals revert 8
[M-03] Incorrect metadata address in get_usdc_init_metadata 9
[M-04] incorrect price conversion in get_total_debt_in_usdc leads to overestimated debt calculations 11
[M-05] Missing recovery path for failed L2 to L1 return transfer can strand borrowed INIT and leave L1 orders stuck 12

Low severity 13

[L-01] Slippage protection effectively disabled on swaps and LP withdrawals 13
[L-02] delta::compound_into_usdc_deposit() is bricked 14
[L-03] RepayAndWithdraw IBC failure leaves “ghost” withdrawal orders in pending_orders 16
[L-04] Incorrect basis points initialization in delta::init leads to permanent borrowing capacity limitation 17
[L-05] USDC depeg assumption can misprice collateral 18
[I-01] Fixed 50/50 LP provisioning may be suboptimal 19
[I-02] Pending amounts not decremented on retry 19
[I-03] Callback ID truncation to u32 can collide after 2^32 operations 20
[I-04] Lack of test suite 20
[I-05] Unsafe default ibc_intermediary can brick inbound hooks and strand funds unless set to the derived intermediate sender 21
[I-06] IBC lifecycle events are incomplete and unit-unspecified 23
[I-07] Borrow IBC tracking stores USDC-value amount while the actual operation is INIT-denominated 24
[I-08] Misleading comment in delta.move#borrow_from_echelon() function about USDC being “sent back on revert” 25

1. About SBSecurity

SBSecurity is a team of skilled smart contract security researchers. Based on the audits conducted and numerous vulnerabilities reported, we strive to provide the absolute best security service and client satisfaction. While it's understood that 100% security and bug-free code cannot be guaranteed by anyone, we are committed to giving our utmost to provide the best possible outcome for you and your product.

Book a Security Review with us at sbsecurity.net or reach out on Twitter @Slavcheww.

2. Disclaimer

A smart contract security review can only show the presence of vulnerabilities **but not their absence**. Audits are a time, resource, and expertise-bound effort where skilled technicians evaluate the codebase and their dependencies using various techniques to find as many flaws as possible and suggest security-related improvements. We as a company stand behind our brand and the level of service that is provided but also recommend subsequent security reviews, on-chain monitoring, and high whitehat incentivization.

3. Risk classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

3.1. Impact

- **High** - leads to a significant loss of assets in the protocol or significantly harms a group of users.
- **Medium** - leads to a moderate loss of assets in the protocol or some disruption of the protocol's functionality.
- **Low** - funds are not at risk.

3.2. Likelihood

- **High** - almost **certain** to happen, easy to perform, or highly incentivized.
- **Medium** - only **conditionally possible**, but still relatively likely.
- **Low** - requires specific state or **little-to-no incentive**.

3.3. Action required for severity levels

- High - **Must** fix (before deployment if not already deployed).
- Medium - **Should** fix.
- Low - **Could** fix.

4. Executive Summary

A time-boxed security review of the **0xRadian/SB_cabal_audit_deltaneutral** repository was conducted by **SBSecurity**. The review was performed by **ret2basic.eth**, **p4y4b13**, **radev_eth**, **Slavcheww** and **blckhv**, who collectively identified **18 issues** in total.

Overview

Project	Cabal Delta-Neutral Vault
Commit Hash	371ebd306756c0c140f874b4e997a20552cb515b
Resolution	85c587b038b983a9d68da070a7f3ba4c2aa91565
Timeline	January 17, 2026 - January 21, 2026

Scope

delta.move, echelon_helper.move, echelon_util.move

Post Audit Condition

The codebase is well-structured and adheres to strong development practices. No Critical or High severity issues were found. Due to the number of recent changes, follow-up audits are recommended, though not mandatory.

5. Findings

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	5
Low/Info Risk	13

ID	Title	Severity	Resolution
[M-01]	`echelon_helper.move#left_to_target_ltv()` function returns inverted rebalance action	Medium	**Fixed**
[M-02]	`delta.move#repay_debt_from_lp_pool_and_withdraw()` sanity check makes partial withdrawals revert	Medium	**Fixed**
[M-03]	Incorrect metadata address in `get_usdc_init_metadata`	Medium	**Fixed**
[M-04]	incorrect price conversion in `get_total_debt_in_usdc` leads to overestimated debt calculations	Medium	**Fixed**
[M-05]	Missing recovery path for failed L2 to L1 return transfer can strand borrowed INIT and leave L1 orders stuck	Medium	**Fixed**
[L-01]	Slippage protection effectively disabled on swaps and LP withdrawals	Low	Acknowledged
[L-02]	`delta::compound_into_usdc_deposit()` is bricked	Low	**Fixed**
[L-03]	`RepayAndWithdraw` IBC failure leaves “ghost” withdrawal orders in `pending_orders`	Low	**Fixed**
[L-04]	Incorrect basis points initialization in delta::init leads to permanent borrowing capacity limitation	Low	**Fixed**
[L-05]	USDC depeg assumption can misprice collateral	Low	**Fixed**
[I-01]	Fixed 50/50 LP provisioning may be suboptimal	Info	Acknowledged
[I-02]	Pending amounts not decremented on retry	Info	**Fixed**

[I-03]	Callback ID truncation to u32 can collide after 2^32 operations	Info	Acknowledged
[I-04]	Lack of test suite	Info	Acknowledged
[I-05]	Unsafe default ibc_intermediary can brick inbound hooks and strand funds unless set to the derived intermediate sender	Info	Acknowledged
[I-06]	IBC lifecycle events are incomplete and unit-unspecified	Info	**Fixed**
[I-07]	Borrow IBC tracking stores USDC-value amount while the actual operation is INIT-denominated	Info	**Fixed**
[I-08]	Misleading comment in `delta.move#borrow_from_echelon()` function about USDC being "sent back on revert"	Info	**Fixed**

Medium Severity

[M-01] `echelon_helper.move#left_to_target_ltv()` function returns inverted rebalance action

Severity: Medium Risk

Description: `left_to_target_ltv()` function is supposed to tell the keeper how to move the position back to the target LTV. But the logic is flipped. It computes `ltv_diff = current_ltv - target_ltv` and when `current_ltv >= target_ltv` (position is over-leveraged), it returns `(0, amount)` meaning "borrow more" instead of "repay". When `current_ltv < target_ltv`, it returns "repay" instead of "borrow".

```
let ltv_diff = bigdecimal::sub(current_ltv, target_ltv);
if (bigdecimal::ge(ltv_diff, bigdecimal::zero())) {
    (0, amount_to_borrow_scaled) // should be repay
} else {
    (1, amount_to_repay_scaled)  // should be borrow
}
```

Impact: The off-chain monitor (per docs) will rebalance in the wrong direction:

- when LTV is too high, it borrows more -> LTV increases -> liquidation risk increases.

- when LTV is too low, it repays -> LTV decreases further -> capital efficiency worsens.

Recommendation: Swap the branches so:

- `current_ltv > target_ltv` -> return "repay"

- `current_ltv < target_ltv` -> return "borrow"

Keep the `(side, amount)` convention consistent with whatever your keeper expects.

Resolution: Fixed

[M-02] `delta.move#repay_debt_from_lp_pool_and_withdraw()` sanity check makes partial withdrawals revert

Severity: Medium Risk

Description: In the partial-withdraw path (`lp_amount_to_withdraw < lp_balance`), the function withdraws only `target_ltv`-scaled LP, removes liquidity into INIT, then converts that INIT back to USDC value and sanity-checks it against the requested `usdc_amount`.

But because you only removed `target_ltv` fraction of the LP, the USDC value of the INIT you got back is roughly `usdc_amount * target_ltv`, not `usdc_amount`. The current ±1% check therefore fails for any realistic target LTV.

```
if (lp_amount_to_withdraw < lp_balance) {
    let true_usdc_amount = convert_init_to_usdc(init_amount);

    // compares usdc_amount to ~usdc_amount * target_ltv
    assert!(usdc_amount * 100 <= true_usdc_amount * 101, 10001);
    assert!(usdc_amount * 100 >= true_usdc_amount * 99, 10002);
}
```

Impact: Partial withdrawals are impossible: the function reverts unless `target_ltv` is ~99%+. It only works when withdrawing the full LP balance (which skips the check).

Recommendation: Compare `true_usdc_amount` against the expected USDC value of the removed LP leg, i.e. `usdc_amount * target_ltv_bps / BPS_BASE` (with a small tolerance), instead of comparing it to `usdc_amount`.

Resolution: Fixed

[M-03] Incorrect metadata address in `get_usdc_init_metadata`

Severity: Medium Risk

Description: The `get_usdc_metadata()` function returns the same address as `get_usdc_init_metadata()`, which is the USDC-INIT liquidity pool address (`@0x543b35a39cfadad3da3c23249c474455d15efd2f94f849473226dee8a3c7a9e1`). However, `get_usdc_metadata()` should return the USDC token metadata address, not the pool address.

The `get_usdc_init_metadata()` function correctly returns the USDC-INIT pool metadata address, but `get_usdc_metadata()` incorrectly returns the pool address instead of the actual USDC token address. This causes issues wherever `get_usdc_metadata()` is used, such as in event emissions and when creating USDC fungible assets.

```
#[view]
public fun get_usdc_metadata(): Object<Metadata> {

object::address_to_object<Metadata>(@0x543b35a39cfadad3da3c23249c474455d15efd2f94f849473226dee8a3c7a9e1)
}

#[view]
public fun get_usdc_init_metadata(): Object<Metadata> {

object::address_to_object<Metadata>(@0x543b35a39cfadad3da3c23249c474455d15efd2f94f849473226dee8a3c7a9e1)
}

fun execute_init_usdc_swap(init_fa: FungibleAsset): FungibleAsset {
    let usdc_init_metadata =
object::address_to_object<Metadata>(@0x543b35a39cfadad3da3c23249c474455d15efd2f94f849473226dee8a3c7a9e1);
    execute_swap(init_fa, usdc_init_metadata)
}
```

Impact: When `get_usdc_metadata()` is called, it returns pool metadata instead of USDC token metadata. This affects:

1. Event emissions that reference USDC metadata.

2. Creation of zero USDC fungible assets (fungible_asset::zero(get_usdc_metadata()))

3. Any other operations that expect USDC token metadata but receive pool metadata instead

This can cause type mismatches, incorrect token references, and potential failures when the code expects token metadata but receives pool metadata.

Proof of Concept:

1. Call `get_usdc_metadata()` expecting USDC token metadata.

2. Observe that it returns the same address as `get_usdc_init_metadata()` (the pool address).

3. Use this metadata in operations expecting USDC token metadata (e.g., `fungible_asset::zero()`, event emissions).

4. The operations may fail or behave incorrectly because pool metadata is being used where token metadata is expected.

Recommendation: Update `get_usdc_metadata()` to return the correct USDC token metadata address, which should be different from the USDC-INIT pool address returned by `get_usdc_init_metadata()`.

Resolution: Fixed

[M-04] incorrect price conversion in `get_total_debt_in_usdc` leads to overestimated debt calculations

Severity: Medium Risk

Description: The `echelon_helper::get_total_debt_in_usdc()` function retrieves `total_liability` from `lending::account_liability_value()`, which according to the developers already returns the value in USDC terms. However, the function incorrectly divides this by the USDC price from the borrow market, inflating the debt value.

The code comment states "Return total liability normalized to USDC, this is not scaled", but since `total_liability` is already in USDC, it should only be converted from Q64.64 format to BigDecimal, not by dividing with the USDC price from borrow market. For example, if the protocol borrowed 100 INIT at INIT = $0.5, the actual debt is $50. `lending::account_liability_value()` correctly returns 50 USDC, but `get_total_debt_in_usdc()` divides by the USDC price (0.5), resulting in 50 / 0.5 = 100 USDC instead of 50 USDC.

```
// Gets the total debt taken out in USDC
#[view]
public fun get_total_debt_in_usdc(): BigDecimal acquires ModuleStore {
    let m_store = borrow_global<ModuleStore>(@delta);
    let asset_store_address = object::address_from_extend_ref(&m_store.asset_store_extend_ref);
    // Given in Q64.64
    let total_liability = fixed_point64::to_u128(lending::account_liability_value(asset_store_address));
    let usdc_conversion = fixed_point64::to_u128(lending::asset_price(m_store.borrow_market));
    // Return total liability normalized to USDC, this is not scaled
    // This division is mathematically sound for two Q64.64 numbers:
    // Q64.64 works like this: X -> X * 2^64, thus (X * 2^64) / (Y * 2^64) = X / Y
    bigdecimal::from_ratio_u128(total_liability, usdc_conversion)
}
```

Similarly `total_collateral` in `get_total_collateral_in_usdc()` is in USDC terms. so the usdc conversion isn't required.

Impact: The overestimated debt affects `get_ltv()` and `left_to_target_ltv()`, causing incorrect LTV calculations and suboptimal rebalancing decisions. The protocol may over-rebalance positions and make unnecessary repayments, reducing capital efficiency.

Proof of Concept:

1. Borrow 100 INIT at $0.5, resulting in $50 debt in USDC terms.

2. `lending::account_liability_value()` returns $50 (already in USDC, Q64.64 format).

3. `get_total_debt_in_usdc()` divides by USDC price (0.5), resulting in $100 instead of $50.

4. `get_ltv()` uses the inflated $100 value, overestimating the LTV ratio.

5. `left_to_target_ltv()` makes incorrect rebalancing decisions based on the wrong debt.

Recommendation: Remove the division by `usdc_conversion` since `total_liability` is already in USDC terms. Convert directly from Q64.64 to BigDecimal.

Resolution: Fixed

[M-05] Missing recovery path for failed L2 to L1 return transfer can strand borrowed INIT and leave L1 orders stuck

Severity: Medium Risk

Description: In the borrow flow, INIT is borrowed on L2 and then sent back to L1 via a separate L2 to L1 IBC transfer (triggered after the borrow). If that return transfer fails or times out, the borrow has already happened on L2, but the INIT never reaches L1 for `delta.move#process_borrow_and_put_into_lp_pool(order_id)` to finalize the order. There is no on-chain retry/rescue mechanism to resend the stranded INIT while preserving the correct `order_id`.

Impact:

1. The position can become cross-chain inconsistent: L2 debt increases, but L1 never receives the borrowed INIT to complete LP provisioning.

2. The associated `pending_orders[order_id]` remains stuck/unfinalizable.

3. Funds can remain stranded on L2 without a clean on-chain recovery path and the `order_id` can be lost.

Recommendation: Add a recovery mechanism for failed L2 to L1 return transfers. Persist enough state on L2 to correlate a failed return with its `order_id` (and amount/metadata) and provide a retry entrypoint that can re-initiate the L2 to L1 transfer and/or re-trigger the L1 processor using the correct `order_id`.

Resolution: Fixed

Low severity

[L-01] Slippage protection effectively disabled on swaps and LP withdrawals

Severity: Low Risk

Description: Several core paths execute swaps or LP withdrawals without enforcing a meaningful minimum output. In `internal_compound_reward_into_usdc()`, xINIT → INIT swaps pass `min_init_out = 0`, and INIT → USDC swaps use `dex::swap` without a min-out parameter. LP withdrawals also pass `option::none<u64>()` for minimum amounts in both `remove_stable_liquidity()` and `single_asset_remove_stable_liquidity()`. This effectively disables slippage protection and exposes the vault to adverse execution or sandwiching during rebalance/compound operations.

Impact: The vault can realize materially worse prices during compounding and debt repayment/withdrawal flows, resulting in performance loss. While rewards may be small at times, these functions can be called during larger rebalances where slippage impact becomes meaningful.

Proof of Concept:

1. Trigger `compound_into_usdc_deposit()` or `repay_debt_from_lp_pool()` while pools are thin.

2. A manipulator shifts pool prices just before the swap/withdraw.

3. Because min-out checks are absent or zero, the vault accepts a worse price and loses value.

Recommendation: Introduce configurable slippage bounds and pass min-out values to swaps and LP withdrawals using pool simulations or oracle-based pricing. Reject execution if outputs fall below the tolerated threshold.

Resolution: Acknowledged

[L-02] `delta::compound_into_usdc_deposit()` is bricked

Severity: Low Risk

Description: `compound_into_usdc_deposit()` deposits swapped USDC into the asset store and then calls `deposit(&asset_signer, usdc_amount)`. The `asset_signer` is a signer derived from the asset store's `ExtendRef` via `object::generate_signer_for_extending`, which represents the object-owned asset store address (not the manager or module address). The `deposit()` entry function requires `assert_is_authorized(signer::address_of(account))` which only allows the `manager_address` or `@delta`. Since the `asset_signer` address is neither, this call always aborts on the authorization check.

Impact: Reward compounding reverts once it tries to re-enter `deposit()`, so rewards cannot be reinvested through `compound_into_usdc_deposit()`. This is a functional break in the intended compounding flow.

Proof of Concept:

1. Call `compound_into_usdc_deposit()` as manager.

2. Function creates `asset_signer` and calls `deposit(&asset_signer, usdc_amount)`.

3. `deposit()` enforces `assert_is_authorized()` and aborts because `asset_signer` is not `manager_address` nor `@delta`.

Recommendation: Add a private helper that performs the supply+borrow flow using the asset store signer (no auth check, no manager transfer), and call it from `compound_into_usdc_deposit()`.

```
// Add this helper in delta.move
fun deposit_from_asset_store(usdc_amount: u64) acquires ModuleStore {
  assert!(usdc_amount > 0, error::invalid_argument(EINVALID_COIN_AMOUNT));

  let m_store = borrow_global_mut<ModuleStore>(@delta);
  let asset_signer = object::generate_signer_for_extending(&m_store.asset_store_extend_ref);

  let init_equivalent_supplied = convert_usdc_to_init_amount(usdc_amount);
  let borrow_ratio = bigdecimal::from_ratio_u64(m_store.target_ltv_bps, BPS_BASE);
  let borrow_init_amount = bigdecimal::mul_by_u64_truncate(borrow_ratio, init_equivalent_supplied);
  let init_metadata = m_store.init_metadata;

  let order_id = register_pending_order(borrow_init_amount);
  let callback_id = register_pending_ibc_operation(
    OperationType::SupplyAndBorrow,
    usdc_amount,
    option::some(order_id)
  );

  echelon_util::request_supply_to_echelon_and_borrow(
    &asset_signer,
    order_id,
    usdc_amount,
    borrow_init_amount,
    callback_id
  );
```

```
  event::emit(VaultActionEvent {
    action: ActionType::Deposit,
    amount: usdc_amount,
    metadata: get_usdc_metadata()
  });
  event::emit(VaultActionEvent {
    action: ActionType::BorrowRequest,
    amount: borrow_init_amount,
    metadata: init_metadata
  });
}
```

Then replace this line in compound_into_usdc_deposit():

```
- deposit(&asset_signer, usdc_amount);
+ deposit_from_asset_store(usdc_amount);
```

Resolution: Fixed

[L-03] `RepayAndWithdraw` IBC failure leaves "ghost" withdrawal orders in `pending_orders`

Severity: Low Risk

Description: In `delta.move#repay_debt_from_lp_pool_and_withdraw()` function, the code creates a withdrawal `order_id` and stores it in `pending_orders`, but tracks the corresponding IBC operation with `order_id = none`. If the IBC call fails or times out, the failure handler can't identify which `pending_orders` entry to remove, so the order is never cleaned up.

```
let callback_id = register_pending_ibc_operation(
    OperationType::RepayAndWithdraw,
    init_amount,
    option::none() // <---- missing order_id
);
let order_id = register_pending_order(usdc_amount); // <---- stored in pending_orders
```

Impact: On IBC failure/timeout, `pending_orders` permanently accumulates stale orders and unbounded growth of `pending_orders`. It also leaves an outstanding withdrawal order that will never "arrive".

Recommendation: Track the `order_id` in the pending IBC operation for `RepayAndWithdraw` (pass `option::some(order_id)`), so failure/timeout handling can reliably clean up the associated `pending_orders` entry.

Resolution: Fixed

[L-04] Incorrect basis points initialization in delta::init leads to permanent borrowing capacity limitation

Severity: Low Risk

Description: The `delta::init()` function sets `target_ltv_bps` to 500 with comments indicating 50%. However, 500 basis points equals 5%, not 50%. Since `BPS_BASE` is 10000, 50% requires 5000 basis points.

The `target_ltv_bps` value is used in `delta::deposit()` to calculate the borrow ratio via `bigdecimal::from_ratio_u64(m_store.target_ltv_bps, BPS_BASE)`, resulting in 0.05 (5%) instead of 0.50 (50%). This limits borrowing to only 5% of the supplied collateral value instead of 50%, severely reducing leverage and reward-earning potential. As there is no setter function for `target_ltv_bps`, this incorrect value becomes permanent after initialization.

Similarly `MAX_PERFORMANCE_FEE_BPS` set to 500 bps which is 5%. But the comments indicating it should be max at 50%. I think which is incorrect as fee shouldn't be capped at 50%.

```
const MAX_PERFORMANCE_FEE_BPS: u64 = 500; // max at 50%

public entry fun init(...) {
    move_to<ModuleStore>(account, ModuleStore {
        target_ltv_bps: 500, // Initiate at 50%
        // ...
    });
```

Impact: Setting `target_ltv_bps` to 5% instead of 50% limits borrowing to only 5% of the supplied collateral value, severely reducing leverage and reward-earning potential. The protocol cannot achieve its intended leverage ratio, resulting in significantly lower returns for users.

Proof of Concept:

1. Call `delta::init()` to deploy the protocol.

2. Call `delta::deposit()` to supply USDC collateral.

3. Observe that only 5% of the collateral value is borrowed as INIT instead of the intended 50%.

Recommendation: Update `target_ltv_bps` to 5000 basis points to correctly represent 50%. This fix must be applied before deployment as these values are immutable.

```
- const MAX_PERFORMANCE_FEE_BPS: u64 = 500; // max at 50%
+ const MAX_PERFORMANCE_FEE_BPS: u64 = 500; // max at 5%

  public entry fun init(...) {
      move_to<ModuleStore>(account, ModuleStore {
-         target_ltv_bps: 500, // Initiate at 50%
+         target_ltv_bps: 5000, // Initiate at 50%
          // ...
```

Resolution: Fixed

[L-05] USDC depeg assumption can misprice collateral

Severity: Low Risk

Description: The strategy implicitly assumes USDC remains at $1. The conversion helpers `convert_usdc_to_init_amount()` and `convert_init_to_usdc()` only query the `init/usd` oracle and treat the input USDC amount as a fixed USD value. If USDC depegs, the system will misprice collateral/borrow sizing and LTV calculations, leading to unexpected leverage or under-collateralization decisions during deposits, borrows, and withdrawals.

Impact: If USDC trades below $1, deposits are overvalued in INIT terms, potentially allowing oversized borrows relative to true USD collateral. If USDC trades above $1, the system becomes overly conservative. This is primarily an economic/market risk rather than a direct bug, but it can degrade safety margins and cause adverse user outcomes during depeg events.

Proof of Concept:

1. Assume USDC trades at $0.90.

2. User deposits 1,000 USDC (market value $900).

3. `convert_usdc_to_init_amount(1_000)` uses `init/usd` only and treats the deposit as $1,000, inflating the INIT-equivalent and borrow sizing by ~11%.

4. Resulting LTV/borrow size exceeds the intended risk target under a depeg scenario.

Recommendation: Since Initia provides a `usdc/usd` feed, it is recommendated to "chain" init/usd with usdc/usd to avoid usdc depeg. This will be a Morpho oracle style chaining, for example computing the inverse of usdc/usd first to get usd/usdc, then mutiply init/usd and usd/usdc to get init/usdc. Be careful with precision loss when computing this inverse, we highly recommend to add extensive test cases to ensure the behavior is correct.

Resolution: Fixed

[I-01] Fixed 50/50 LP provisioning may be suboptimal

Severity: Information Risk

Description: `process_borrow_and_put_into_lp_pool()` splits the borrowed INIT 50/50: half is converted to xINIT and then both tokens are provided to the xINIT/INIT LP. This fixed ratio ignores pool state, price deviations, and optimal LP deposit ratios, which can increase slippage or leave residuals.

Impact: Over time, deposits may incur unnecessary slippage or leave value on the table versus a ratio based on pool composition or swap pricing. This reduces yield efficiency and can mildly degrade vault performance.

Proof of Concept:

1. Assume the pool is imbalanced or xINIT is trading at a premium/discount.

2. The 50/50 split forces a non-optimal deposit ratio, causing extra swap losses or lower LP shares.

Recommendation: Compute the optimal deposit ratio based on pool reserves or price, and swap only the needed amount to match that ratio before providing liquidity. Consider using pool simulation helpers if available.

Resolution: Acknowledged

[I-02] Pending amounts not decremented on retry

Severity: Information Risk

Description: On IBC failure or timeout, `handle_ibc_failure()` increments `m_store.collateral_pending_amount` or `m_store.init_pending_amount`. There is no mechanism to decrement these fields when a retry succeeds or when funds are recovered, so even if a future retry path is implemented, the pending counters will remain inflated and desynchronized from actual balances.

Impact: Incorrect pending accounting can break operator tooling, misreport available balances, and cause follow-up logic that relies on these fields to malfunction or make unsafe decisions.

Proof of Concept:

1. Trigger an IBC failure so `collateral_pending_amount` or `init_pending_amount` is incremented.

2. Retry the operation off-chain or via a hypothetical recovery path that succeeds.

3. The pending counters remain incremented, leaving permanent overcounting.

Recommendation: When implementing retry or recovery, decrement `collateral_pending_amount`/`init_pending_amount` upon successful completion. Consider tracking pending amounts per callback/order so they can be reconciled precisely rather than using aggregate counters.

Resolution: Fixed

[I-03] Callback ID truncation to u32 can collide after 2^32 operations

Severity: Information Risk

Description: Both L1 (`delta::echelon_util`) and L2 (`delta::echelon_helper1`) track `callback_id` as `u64`, but serialize it into the async callback memo as a `u32` (`id: callback_id as u32`). This truncation means that after 2^{32} callbacks, IDs will wrap and collide, causing acknowledgements/timeouts to map to the wrong pending operation.

Impact: In a long-lived deployment with very high throughput, callback ID collisions can corrupt IBC operation tracking, mis-handle failures, and potentially strand funds or desync accounting.

Proof of Concept:

1. Observe that `callback_id` is incremented as `u64` on both L1 and L2.

2. In the memo builder, `callback_id` is cast to `u32` for `async_callback.id`.

3. After 2^{32} operations, the serialized `id` repeats and collides with a prior pending entry.

Recommendation: Serialize the full `u64` callback ID in async callbacks (or enforce a strict cap that prevents wraparound). If a `u32` is required by downstream logic, rotate/renew channels or reset state before 2^{32} operations.

Resolution: Acknowledged

[I-04] Lack of test suite

Severity: Information Risk

Description: The repository does not include a runnable test suite that exercises core protocol flows (deposit, borrow, repay, withdraw, IBC hooks). There are no Move unit tests under `sources/` or a dedicated `tests/` directory.

Impact: Bugs in cross-chain logic, accounting, and authorization are harder to detect before deployment. Regressions may go unnoticed and fixes are less verifiable.

Recommendation: Add a minimal Move test suite that covers:

- L1 manager flows (`deposit`, `borrow_from_echelon`, `repay_debt_from_lp_pool`, `withdraw_collateral`).

- L2 helper flows (`supply_to_echelon_and_borrow`, `repay_and_withdraw`).

- IBC hook callbacks (ibc_ack, ibc_timeout) and failure paths.

- Misconfiguration cases (wrong channel, wrong intermediary).

Resolution: Acknowledged

[I-05] Unsafe default ibc_intermediary can brick inbound hooks and strand funds unless set to the derived intermediate sender

Severity: Information Risk

Description: On Initia, the Move IBC-hooks middleware executes the receiver-side hook with a derived intermediate sender (based on `destChannel` + original packet sender), not the IBC module account.

However, `delta::echelon_helper1` gates its hook entrypoints by checking that the signer equals a stored `ibc_intermediary` address:

- `supply_to_echelon_and_borrow()` and `repay_and_withdraw()` call `assert_is_ibc_intermediary(signer::address_of(account))`.
- `ModuleStore.ibc_intermediary` is initialized to `@0x1` → based on the derivation algorithm we found in Initial IBC move hooks source code, the intermediate sender will never be `@0x1`.

The derivation algorithm is defined by Initia's chain implementation of Move IBC hooks:

- In Initia's `x/ibc-hooks/move-hooks/util.go`, the function `DeriveIntermediateSender(channel, originalSender)` constructs a string like `"<channel>/<originalSender>"`, hashes it with a fixed prefix, and returns the bech32 account address of that hash.
    - Source: https://github.com/initia-labs/initia/blob/main/x/ibc-hooks/move-hooks/util.go
- In Initia's `x/ibc-hooks/move-hooks/receive.go`, the receive-path code calls:
    - `DeriveIntermediateSender(packet.GetDestChannel(), data.GetSender())` and then executes the Move hook with that derived value as the sender.
    - Source: https://github.com/initia-labs/initia/blob/main/x/ibc-hooks/move-hooks/receive.go

As a result, the effective hook signer is deterministic but depends on both the destination channel and the packet's original sender.

The severity of this finding downgrades to Low because there is an admin-only setter `set_ibc_intermediary()` (gated to `@delta`) that can update this value.

Impact:

- **Bricking / stranded funds on first use:** If the system sends L1→L2 packets before `ibc_intermediary` is set correctly, the ICS-20 receive can succeed (funds minted/credited to the intermediate sender) but the Move hook aborts and the acknowledgement is an error.
- The funds can then be **stuck at the intermediate sender address** on L2 (an address with no private key), with no on-chain recovery in this code.

Recommendation: If keeping the current pattern, make the deployment procedure explicitly set `ibc_intermediary` before any IBC operation, to the expected derived intermediate sender.

Prefer making the contract self-configuring and safer-by-default:

- store trusted channel + trusted remote sender, and
- derive/validate the effective signer (or otherwise validate packet provenance) in a way that matches ibc-hooks.

Resolution: Acknowledged

[I-06] IBC lifecycle events are incomplete and unit-unspecified

Severity: Information Risk

Description:

1. `IBCFailureEvent` in `delta.move` emits `amount` from `IbcOperation.amount`, but that field isn't consistently denominated across operations (e.g., `Borrow` stores a USDC-denominated amount while the underlying asset is INIT; `RepayAndWithdraw` failure emits only the INIT leg and doesn't capture the USDC withdraw leg). Off-chain consumers must special-case by `operation_type` to interpret it correctly.

2. The L1 settlement handlers that finalize inbound IBC funds don't emit any events:

- `process_borrow_and_put_into_lp_pool(orderId)` (order removed, INIT->xINIT, LP minted, LP deposited)
- `process_collateral_withdraw(orderId)` (order removed, collateral moved into withdrawal store)

So indexers/keepers can see "request" events, but don't get a clean "settled/processed" signal without reading state diffs.

Impact:

1. Off chain observers can't reliably interpret amount without special-casing `operation_type`.

2. Off chain sees when a request has been made but doesn't get an explicit "settled" signal when the return leg succeeded and was processed.

Recommendation:

- Make `IBCFailureEvent` specific by including `callback_id`, `order_id` and the asset/denom.

- Emit explicit "settled" events in `delta.move#process_borrow_and_put_into_lp_pool()` and `delta.move#process_collateral_withdraw()` functions (include `orderId` and the finalized amounts/metadata).

Resolution: Fixed

[I-07] Borrow IBC tracking stores USDC-value amount while the actual operation is INIT-denominated

Severity: Information Risk

Description: In `delta.move#borrow_from_echelon()` function, the code tracks the pending order in INIT units (`pending_orders[order_id].amount = init_amount`), but tracks the pending IBC operation for `Borrow` using the USDC-value input (`pending_ibc_operations[callback_id].amount = usdc_amount`). This makes the stored `amount` field incorrect for `OperationType::Borrow`.

Same for:

- `pending_ibc_operations.amount` for `RepayAndWithdraw` is INIT, while the created `pending_orders.amount` is USDC

- in `withdraw_collateral`, the code stores `pending_ibc_operations.amount = usdc_amount` and (on failure) `IBCFailureEvent` will emit `amount = usdc_amount` for `OperationType::Withdraw`.

Impact: Makes the `IBCFailureEvent.amount` incorrect (it will emit `usdc_amount` even though the failed operation is "borrow INIT").

Recommendation: Make the tracking consistent and self-describing. For example, store `init_amount` for `Borrow` or include a metadata/denomination field (or both USDC-value + INIT amount) so events and indexers can interpret the value correctly.

Resolution: Fixed

[I-08] Misleading comment in `delta.move#borrow_from_echelon()` function about USDC being "sent back on revert"

Severity: Information Risk

Description: The comment says the pending tx "should be in USDC because if it reverts, the USDC is sent back". In the borrow-only path (`supply_amount = 0`), the code does not send USDC at all; it mints and transfers 1 DELTA token as an IBC carrier for the memo.

```
// If we only borrow, then there is no token to send over so we need the delta token
coin::mint_to(&m_store.delta_token_caps.mint_cap, signer::address_of(account), 1);
cosmos::transfer(account, receiver, m_store.delta_token_metadata, 1, ...);
```

Impact: Invalid and incorrect comment about what is actually being transferred/refunded on.

Recommendation: Update the comment to match the real behavior: borrow-only uses a 1-unit DELTA carrier token, not USDC and refunds apply to that carrier token.

Resolution: Fixed